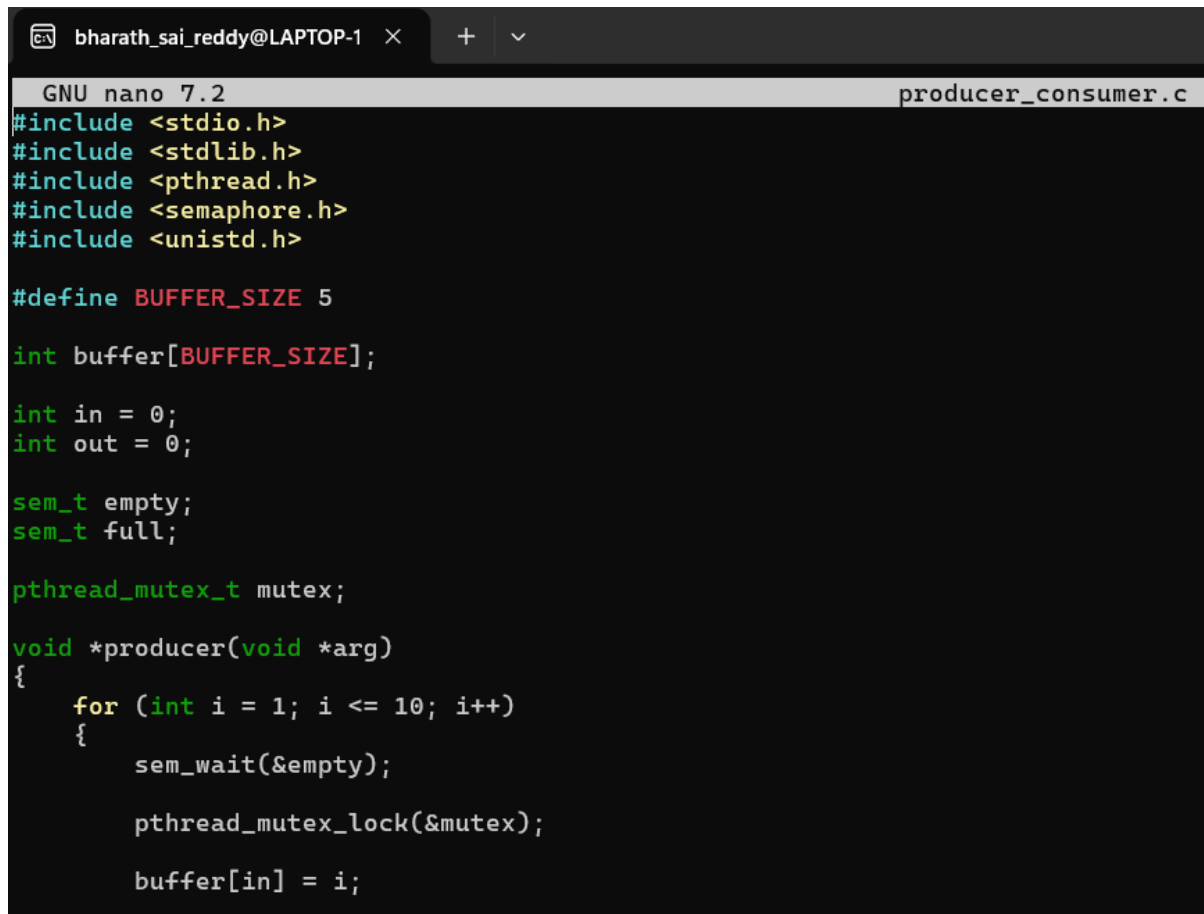


Practical -12

1) Producer consumer



```
GNU nano 7.2 producer_consumer.c
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define BUFFER_SIZE 5

int buffer[BUFFER_SIZE];

int in = 0;
int out = 0;

sem_t empty;
sem_t full;

pthread_mutex_t mutex;

void *producer(void *arg)
{
    for (int i = 1; i <= 10; i++)
    {
        sem_wait(&empty);

        pthread_mutex_lock(&mutex);

        buffer[in] = i;
```

```
    printf("Producer produced: %d\n", i);

    in = (in + 1) % BUFFER_SIZE;

    pthread_mutex_unlock(&mutex);

    sem_post(&full);

    sleep(1);
}

return NULL;

void *consumer(void *arg)
{
    for (int i = 1; i <= 10; i++)
    {
        sem_wait(&full);

        pthread_mutex_lock(&mutex);

        int item = buffer[out];

        printf("Consumer consumed: %d\n", item);

        out = (out + 1) % BUFFER_SIZE;
```

bharath_sai_reddy@LAPTOP-1 × + v
GNU nano 7.2 producer_consumer.c

```
        pthread_mutex_unlock(&mutex);  
        sem_post(&empty);  
        sleep(2);  
    }  
    return NULL;  
}  
  
int main()  
{  
    pthread_t producer_thread;  
    pthread_t consumer_thread;  
  
    sem_init(&empty, 0, BUFFER_SIZE);  
    sem_init(&full, 0, 0);  
  
    pthread_mutex_init(&mutex, NULL);  
  
    pthread_create(&producer_thread, NULL, producer, NULL);  
    pthread_create(&consumer_thread, NULL, consumer, NULL);  
  
    pthread_join(producer_thread, NULL);  
    pthread_join(consumer_thread, NULL);  
  
    sem_destroy(&empty);
```

```
    sem_destroy(&full);  
    pthread_mutex_destroy(&mutex);  
  
    return 0;  
}
```

```
bharath_sai_reddy@LAPTOP-1  ×  +  ▾
.gitkeep      Practical-02 Practical-04 Practical-06 Practical-08 Practical-10 practical-12
Practical-01 Practical-03 Practical-05 Practical-07 Practical-09 practical-11
bharath_sai_reddy@LAPTOP-1711DF88:~/90128_OSSP/2520090128_Practical$ ls
Practical-01 Practical-03 Practical-05 Practical-07 Practical-09 practical-11
Practical-02 Practical-04 Practical-06 Practical-08 Practical-10 practical-12
bharath_sai_reddy@LAPTOP-1711DF88:~/90128_OSSP/2520090128_Practical$ cd practical-12
bharath_sai_reddy@LAPTOP-1711DF88:~/90128_OSSP/2520090128_Practical/practical-12$ nano producer_consumer.c
bharath_sai_reddy@LAPTOP-1711DF88:~/90128_OSSP/2520090128_Practical/practical-12$ gcc producer_consumer.c -pthread
bharath_sai_reddy@LAPTOP-1711DF88:~/90128_OSSP/2520090128_Practical/practical-12$ ./producer_consumer
Producer produced: 1
Consumer consumed: 1
Producer produced: 2
Consumer consumed: 2
Producer produced: 3
Producer produced: 4
Consumer consumed: 3
Producer produced: 5
Producer produced: 6
Consumer consumed: 4
Producer produced: 7
Producer produced: 8
Consumer consumed: 5
Producer produced: 9
Producer produced: 10
Consumer consumed: 6
Consumer consumed: 7
Consumer consumed: 8
Consumer consumed: 9
Consumer consumed: 10
```

2) deadlock

GNU nano 7.2

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t resource1;
pthread_mutex_t resource2;

void *thread1(void *arg)
{
    printf("Thread 1: locking Resource 1\n");
    pthread_mutex_lock(&resource1);

    sleep(1);

    printf("Thread 1: waiting for Resource 2\n");
    pthread_mutex_lock(&resource2);

    printf("Thread 1: acquired both resources\n");

    pthread_mutex_unlock(&resource2);
    pthread_mutex_unlock(&resource1);

    return NULL;
}

void *thread2(void *arg)
```

```
bharath_sai_reddy@LAPTOP-1 × + v
GNU nano 7.2 deadlock.c
{
    printf("Thread 2: locking Resource 2\n");
    pthread_mutex_lock(&resource2);

    sleep(1);

    printf("Thread 2: waiting for Resource 1\n");
    pthread_mutex_lock(&resource1);

    printf("Thread 2: acquired both resources\n");

    pthread_mutex_unlock(&resource1);
    pthread_mutex_unlock(&resource2);

    return NULL;
}

int main()
{
    pthread_t t1, t2;

    pthread_mutex_init(&resource1, NULL);
    pthread_mutex_init(&resource2, NULL);

    pthread_create(&t1, NULL, thread1, NULL);
    pthread_create(&t2, NULL, thread2, NULL);
```

```
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    return 0;
}
```

```
bharath_sai_reddy@LAPTOP-1711DF88:~/90128_OSSP/2520090128_Practical/practical-12$ nano deadlock.c
bharath_sai_reddy@LAPTOP-1711DF88:~/90128_OSSP/2520090128_Practical/practical-12$ gcc deadlock.c -o deadlock -pthread
bharath_sai_reddy@LAPTOP-1711DF88:~/90128_OSSP/2520090128_Practical/practical-12$ ./deadlock
Thread 2: locking Resource 2
Thread 1: locking Resource 1
Thread 2: waiting for Resource 1
Thread 1: waiting for Resource 2
^C
```

3) deadlock prevention

```
bharath_sai_reddy@LAPTOP-1 × + v
GNU nano 7.2                                deadlock_prevention.c
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t resource1;
pthread_mutex_t resource2;

void *thread1(void *arg)
{
    printf("Thread 1: locking Resource 1\n");
    pthread_mutex_lock(&resource1);

    sleep(1);

    printf("Thread 1: locking Resource 2\n");
    pthread_mutex_lock(&resource2);

    printf("Thread 1: acquired both resources\n");

    pthread_mutex_unlock(&resource2);
    pthread_mutex_unlock(&resource1);

    return NULL;
}

void *thread2(void *arg)
```

```
bharath_sai_reddy@LAPTOP-1 × + v
GNU nano 7.2 deadlock_prevention.c
{
    printf("Thread 2: locking Resource 1\n");
    pthread_mutex_lock(&resource1);

    sleep(1);

    printf("Thread 2: locking Resource 2\n");
    pthread_mutex_lock(&resource2);

    printf("Thread 2: acquired both resources\n");

    pthread_mutex_unlock(&resource2);
    pthread_mutex_unlock(&resource1);

    return NULL;
}

int main()
{
    pthread_t t1, t2;

    pthread_mutex_init(&resource1, NULL);
    pthread_mutex_init(&resource2, NULL);

    pthread_create(&t1, NULL, thread1, NULL);
    pthread_create(&t2, NULL, thread2, NULL);
|
```

```
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    pthread_mutex_destroy(&resource1);
    pthread_mutex_destroy(&resource2);

    return 0;
}
-|
```

```
bharath_sai_reddy@LAPTOP-1711DF88:~/90128_OSSP/2520090128_Practical/practical-12$ ./deadlock_prevention
Thread 1: locking Resource 1
Thread 2: locking Resource 1
Thread 1: locking Resource 2
Thread 1: acquired both resources
Thread 2: locking Resource 2
Thread 2: acquired both resources
```